PROJECT OVERVIEW

GlacierWatch

A Streamlit app that screens for glacial lake outburst flood (GLOF) risk: it measures how lakes change in satellite images, scores them, finds the villages downstream, and prepares email alerts.

Project folder: glacier_lake / clean | Report date: 1 November 2026 | Language: Python 3.14, run with `uv`

6	5	2	5
Python modules (about 1,470 lines)	App pages in the sidebar menu	YOLO segmentation models	External services used

1 What the project does

Glacial lakes in the Himalaya can grow quickly as glaciers melt. If a lake's natural dam fails, the water can flood valleys and villages far downstream. GlacierWatch is an **early-warning screening tool**: it looks at satellite images of lakes, measures how much each one has grown, combines that with other risk indicators into a 0-100 score, works out which villages lie along the river below, and prepares alert emails for the people responsible for them. The example it was built around is South Lhonak Lake in Sikkim, but the Region Monitoring page accepts any region name.

Everything it produces is a *screening* result, not a confirmed flood warning. The email text says so explicitly, and emails are never sent without the user typing `SEND`.

The two main ways to use it

1

Change Detection

Upload an earlier and a later satellite photo of the same place. A YOLO model outlines the lake (or glacier ice) in each. The app compares the two outlines: area that stayed, area gained, area lost.

Gives a 0-10 risk score. Above 1.0 (a change greater than 10%) counts as **Risky** and can email the subscriber list automatically.

Also writes a report, answers typed or spoken questions, and sends alert emails.

2

Region Monitoring

Type a region name. The app finds the place, locates its glacial lakes from recent Sentinel-2 imagery, then fetches Landsat images from a "before" year and an "after" year for every lake and runs YOLO on each.

Each lake gets a 0-100 score (Low / Medium / High). The app finds villages downstream, writes one summary report, and plans emails to registered contacts.

Region Monitoring, step by step

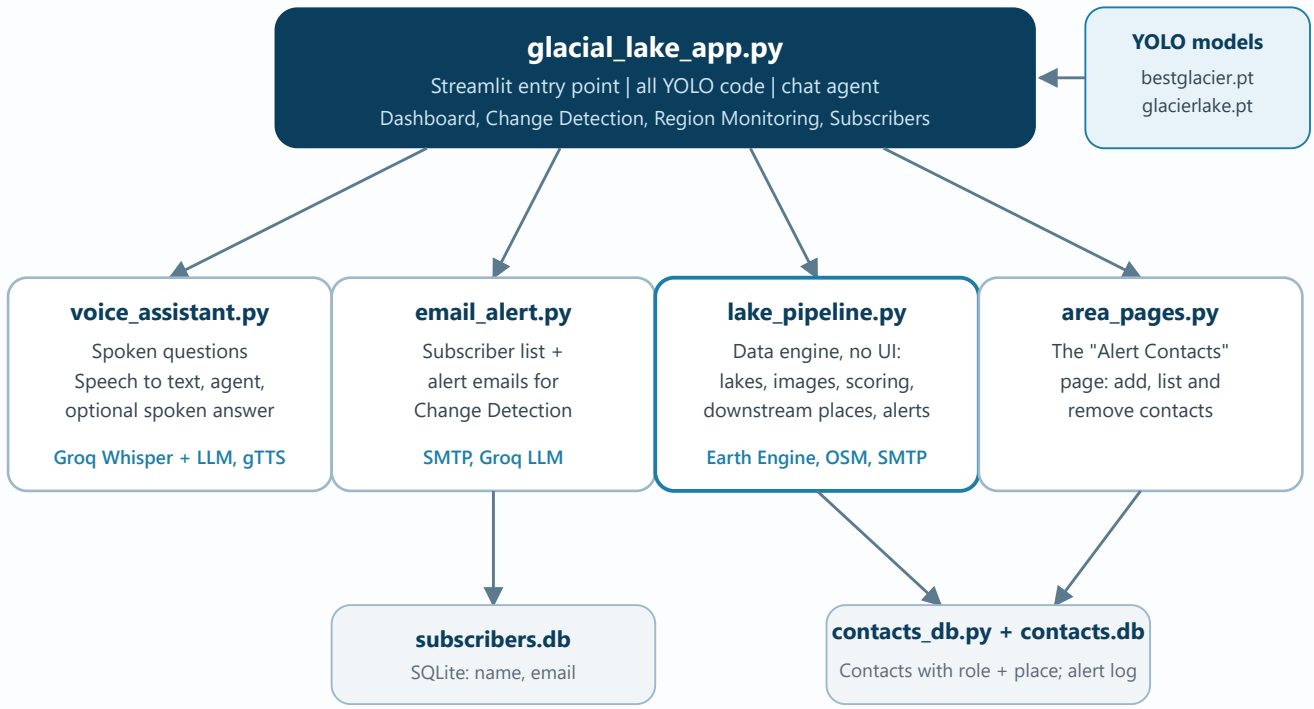
1. **Locate:** the region name is geocoded with OpenStreetMap Nominatim (search circle up to 20 km).
2. **Find lakes:** Sentinel-2 water detection in Google Earth Engine (water index above 0.05, near-infrared below 1500, slope below 15 degrees, at least 1 ha), matched to the 1990-2020 lake inventory and named from OpenStreetMap. The largest N lakes are kept (default 8, up to 20).

3. **Measure change:** for each lake, a Landsat true-colour image for each chosen year is downloaded (Landsat 5, 7, 8 or 9 depending on the year). YOLO outlines the lake in both images and the area change is computed in hectares.
4. **Score:** change (40) + lake size (20) + long-term inventory growth (20) + lake type (20) = 0-100.
5. **Downstream places:** the river basin chain below each lake (HydroSHEDS, up to 120 km) is intersected with OpenStreetMap villages that are at least 100 m lower than the lake and within 60 m of the river's height.
6. **Report and alerts:** only after every lake is done, one summary report is written. Authorities receive it if any lake is Medium or High. Residents of a downstream village get a short notice if a High lake is above them. The default is a dry run.

Low	Medium	High
score below 30	score 30 to 59	score 60 or more

2 How the pieces fit together

The app is one Streamlit program with a thin set of helper modules. `glacial_lake_app.py` draws every page and holds all the YOLO code. The satellite and data work lives in `lake_pipeline.py`, which has no user interface. Email recipients are stored in two separate SQLite databases, one for each alert path.



Arrows show "uses". The thicker outline marks the largest module and the heart of Region Monitoring.

Why two alert paths? Change Detection was built first and emails one flat list of subscribers (`email_alert.py`). Region Monitoring came later and needed to know who is an authority and who is a resident of which village, so it uses a richer contacts table (`contacts_db.py`). The two lists do not share data (see finding B1 in section 4).

3 File-by-file guide

Only the project's own files are listed. The `.venv` folder (about 30,000 files) is the Python environment and is covered at the end of the table.

APPLICATION CODE

File	Purpose and why it exists	How it fits
glacial_lake_app.py 602 lines, 32 KB	**The entry point.** Draws every page, applies the shared styling, loads the YOLO models, runs the image comparison (mask prediction, area change, overlays), builds the Groq chat agent that writes reports and answers questions, and triggers alert emails. Run it with `uv run streamlit run glacial_lake_app.py`.	Calls everything else. Imports `email_alert`, `voice_assistant` and `area_pages` directly, and imports `lake_pipeline` only when Region Monitoring opens, so the rest of the app still starts if Earth Engine is unavailable.
lake_pipeline.py 517 lines, 25 KB	**The satellite and data engine** with no user interface. Detects lakes from Sentinel-2 in Earth Engine, reads the 1990-2020 lake inventory shapefiles, traces rivers downstream (HydroSHEDS) and finds villages (OpenStreetMap). Also geocodes region names, downloads Landsat before/after images, computes the 0-100 YOLO-based score, writes the region report and plans/sends the emails.	Used only by the Region Monitoring page. Reads contacts through `contacts_db`. Has no YOLO code itself; the app passes YOLO results back in for scoring.
area_pages.py 41 lines, 2 KB	Holds the **"Alert Contacts" page**: a form to add authorities and residents, and a list with remove buttons. It also suggests village names taken from the last Region Monitoring result.	Plugged into the app's menu. Writes to `contacts.db` through `contacts_db`. The name is a leftover (see B5).
contacts_db.py 83 lines, 3 KB	**SQLite storage** for contacts (name, email, role, place) and an alert log. The log stops the same person receiving the same alert twice within 24 hours.	Used by `lake_pipeline.py` (to plan and deliver emails) and `area_pages.py` (to manage contacts).
email_alert.py 129 lines, 6 KB	The **simpler, original alert path**: stores the subscriber list in `subscribers.db`, sends SMTP email (it only prints a "dry run" line if SMTP is not configured), and can let a Groq LLM write the alert text and send it through a tool. A plain no-LLM fallback exists.	Used by Change Detection and by the Dashboard/Subscribers pages.
voice_assistant.py 98 lines, 5 KB	**Voice questions.** Records audio, transcribes it with Groq Whisper (dropping the phantom phrases Whisper invents for silence), answers through a small LangChain agent, and optionally speaks the answer with gTTS.	Called from the Change Detection page when a Groq key is set. Standalone logic with its own lookup tool.

MODELS, DATA AND CONFIGURATION

File	Purpose and why it exists	How it fits

bestglacier.pt 24 MB	YOLO segmentation model whose class is **glacial lake**.	Used for Region Monitoring and for the "Glacial lake" option in Change Detection. The file name does not match its contents (see B4).
glacierlake.pt 55 MB	YOLO segmentation model whose class is **glacier ice**.	Used for the "Glacier ice" option in Change Detection only.
pyproject.toml 435 bytes	Declares the project and its 13 dependencies (Streamlit, Ultralytics, OpenCV, LangChain/Groq, Earth Engine and others).	Read by `uv` to build the environment.
uv.lock 456 KB	Exact pinned versions of every dependency, so the environment is reproducible.	Generated by `uv`; keep it with `pyproject.toml`.
.env 216 bytes	Secrets and settings: `GROQ_API_KEY`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_HOST`, `SMTP_PORT`, `FROM_EMAIL`. Values were not read for this report.	Loaded by every module at start-up. Without SMTP values, emails stay in dry-run mode.
contacts.db 20 KB, empty	Auto-created by `contacts_db.py`. Tables: `contacts`, `alerts_log`. Currently 0 rows.	Region Monitoring recipient list.
subscribers.db 8 KB, empty	Auto-created by `email_alert.py`. Table: `subscribers`. Currently 0 rows.	Change Detection recipient list.
README.md 3.7 KB	Short description of the app, how to run it, what each file does, and a log of files removed during earlier clean-ups.	Documentation only. It has been updated to match the current state.

GENERATED OR ENVIRONMENT FOLDERS

Item	What it is	Needed?
.venv/	The Python virtual environment, about 30,000 files of installed packages.	Yes, but never edit it by hand; `uv` manages it.
__pycache__/	Compiled `.pyc` files Python writes automatically (6 files).	No. Safe to delete; it regenerates on the next run.
.codemie/	An empty folder created by the CodeMie tooling, not by the project.	No. Safe to ignore or delete.
osm_places_*.json created at run time	Cache of OpenStreetMap villages, written by Region Monitoring so it does not re-download them. None are in the folder right now.	Optional. Rebuilt on demand.
GlacialLake_20220726/ not in the folder	The 1990-2020 glacial lake inventory shapefiles the scoring relies on.	**Yes, and it is missing.** See finding A1.

4 Unused, unclear and risky items

Findings are grouped by how much they matter. Items marked *done* were already handled during this review.

A. Needs attention

ACTION **A1. The lake inventory folder is missing**

The README and `lake_pipeline.py` expect a folder named `GlacialLake_20220726/` containing `GlacialLake_1990.shp` … `GlacialLake_2020.shp` (plus their `.prj` files). It is not in this folder. Without it the code quietly finds no history, so every lake is labelled "not in 2020 inventory" and gets neutral default points for growth history and lake type instead of real ones. Scores still appear, but they are less meaningful.

Fix: copy the folder back next to `lake_pipeline.py`.

SECURITY **A2. Live secrets sit in a cloud-synced folder with no ignore file**

`.env` holds an API key and email credentials, and the project lives under OneDrive, so those secrets are synced to the cloud. The folder is not a git repository and has no `.gitignore`, so a future `git init` followed by `git add .` would commit them.

Fix: add a `.gitignore` with `.env`, `.venv/` and `__pycache__/` before using git, and consider moving the project out of OneDrive or rotating the credentials if the folder has been shared.

B. Worth fixing

DESIGN **B1. Two separate recipient lists**

The Subscribers page (`subscribers.db`) feeds Change Detection. The Alert Contacts page (`contacts.db`) feeds Region Monitoring. Adding someone to one does not add them to the other, which is easy to forget.

Option: keep one contacts table and give subscribers a default role.

DUPLICATION **B2. Two different email senders**

`email_alert.send_one_email` prints a dry-run line when SMTP is missing and adds a greeting. `lake_pipeline.send_email` raises an error instead and uses a 30-second timeout and certificate checks. They do the same job with different behaviour.

CONFIG **B3. `pandas` is used but not declared**

`glacial_lake_app.py` imports pandas, but `pyproject.toml` does not list it. It works only because Streamlit installs it as a side effect.

Fix: run `uv add pandas`.

NAMING **B4. The two model file names look swapped**

`bestglacier.pt` detects **lakes** and `glacierlake.pt` detects **ice**. The code and README are consistent with each other, but a newcomer will assume the opposite.

Option: rename them (for example `lake_model.pt` and `ice_model.pt`) and update the `MODELS` dictionary in `glacial_lake_app.py`.

NAMING **B5. `area_pages.py` no longer matches its contents**

It used to hold the Area Risk Screening page. That feature was removed, so the file now holds only the Alert Contacts page, and the function is still called `page_area_contacts`.

Option: rename it to `contact_pages.py` and update the import in the app.

PORTABILITY **B6. Hard-coded cloud project and a very new Python requirement**

`lake_pipeline.py` has a Google Cloud project ID written into the code, so only users authenticated to that project can run Earth Engine. `requires-python = ">=3.14"` is also very recent and may block some packages.

DUPLICATION **B7. Repeated settings and lookup tools**

The default Groq model name is defined in three files, and the app and `voice_assistant.py` each define a near-identical "look up a measurement" tool. This is harmless but means a change must be made in several places.

C. Minor or informational

INFO **C1. Never-used database features**

`contacts_db.list_contacts()` accepts a `place` filter that nothing calls, and the `active` column is always 1 because nothing in the UI can deactivate a contact.

INFO **C2. Safe-to-delete folders**

`__pycache__/` (regenerates automatically) and the empty `.codemie/` folder (not part of the project). A delete command for `__pycache__/` was blocked earlier in this session, so it is still there.

CORRECTION **C3. Alert threshold of 1.0 is consistent**

An earlier review in this session called `ALERT_SCORE = 1.0` "very low". That was wrong. The Change Detection score is the absolute percentage change divided by 10, so a score above 1.0 means a change above 10%, which is exactly the "Risky" threshold (`RISK_LIMIT = 10`). The two settings agree. The 10% limit itself is sensitive, since normal differences between two images can exceed it.

D. Already cleaned up during this review

DONE **Area Risk Screening feature removed**

Its page, menu entry, scoring and baseline code, command-line entry point and chat tool (`lake_tool.py` , deleted) are gone. Everything Region Monitoring and Change Detection use was kept.

DONE **Unused dependencies removed**

`fastapi` and `uvicorn` were removed from `pyproject.toml` and `uv.lock` with `uv remove` . Every remaining dependency is imported by the code.

5 Running the project

Need	Detail
Start the app	`uv run streamlit run glacial_lake_app.py`

AI report, chat and voice	Set `GROQ_API_KEY` in `.env`. Without it the app still works and writes a plain-text report.
Real emails	Set `SMTP_USER` and `SMTP_PASSWORD` (for Gmail, an app password). Otherwise everything is a dry run. Region Monitoring additionally needs `SEND` typed to confirm.
Satellite features	Google Earth Engine access for the configured cloud project, plus internet access for OpenStreetMap lookups.
Scoring data	The `GlacialLake_20220726/` inventory folder (currently missing).

Prepared from a read-through of every project file. Secret values in `.env` were deliberately not read. Line counts and sizes are as of 1 November 2026.